Overall Feedback:While each arrow has its corresponding relationship symbol, however adding the actual relationship helps to clear up any formatting differences. This can be applicable to arrows that do not use the default UML format and helps with the overall readability of the diagram, especially when multiple classes are interacting with one another.

Additionally, it is useful to both the reader and the programmer to see the input arguments and return types for each method/constructor. When multiple classes are interacting with one another, it should show how these classes relate. For example, if object A collects object B (aggregation) then it must show object B in the field. This improves readability as well as show the boundaries of the relationship.

Suggestion: The game class uses pieces as objects, and will be destroyed when game object ends. To better capture this relationship a stronger from of association is better suited, possibly composition or aggregation.

Feedback: Since there are various different games that require the same features such as start game, take turn, and reset game, Game class should be abstract. By making the class abstract, other games can extend the class and tailor to their personal logic while keeping the functionality the same.

Feedback: Unclear how types of pieces are assigned, it would be ideal to use piece as an abstract class where there can be class that extend this class for each game. Additionally, enumeration can be used here where the piece constructor takes in an enum and sets its type.

Suggestion: checkWinCondition() and checkTieCondition() can be moved to GameLogic class where it is more appropriate. Rather than each game having different WinCondition(), it is better suit to have extensions of GameLogic since game for the most part are similar, however logic is different.

Feedback: Unclear how multiple players will interact with the board, the diagram only shows a single player interaction. May use queues along with sequence assigning to ensure each player gets their turn. Ideally this would have one to many relationship with the game since multiple players are active during game play.

Suggestion: The game class uses pieces as objects, and will be destroyed when game object ends. To better capture this relationship a stronger from of association is better suited, possibly composition or aggregation.

Feedback: Composition with multiplicity is not ideal here since user can have one curated profile. Having multiple profiles does not enhance the functionality of the application and can instead create new logins. In this case the multiplicity should be removed to have 1...1 relationship.

Suggestion: Display methods should be part of the GUI rather than the back end portion of the profile. However, methods such as displayStats() can return a string of data that can be passed to the GUI to be printed, which would require the method to have a return type of string.

Feedback: There is conflicting information regarding the rank, player class on the right it shows that rank is a enum while here it shows as a string. Having consistency across the diagram is key to ensure all classes interact with other classes in the same manner.

Suggestion: EmailService should be connected to the networking class to ensure that the class is utilized appropriately. Without having any network communication it is difficult to transmit data for confirmation and resetting password.

Suggestion: EmailService will require authentication to prove the email exists, and/or any other proof of identity to ensure the user is authorized to change credential. This can either be integrated within the class using helper methods, or can use AuthenticationService in which case relationship be reversed.

Suggestion: viewCurrentProfile() can be connected to user stats that shows all the details. Rather than having a method that performs similar functions as a class it is beneficial to use the UserStats with an association.

Suggestion: CurrentStatus can be changed into a enum which will be set when creating the profile or toggled by the user. Having a string type is not ideal when setting the status for accessibility and may require additional error checking.

Suggestion: Rank is set in the profile, rather than having a field with the same attribute, it is best to reduce redundancy. Can also implement a method in user that can get all field attributes to set userstats.

Suggestion: GameHistory is stored into an arraylist, which can be better captured using association with multiplicity. This shows that these objects will tracked simultaneously.

Suggestion: Composition arrow should be reversed, player class owns rank objects. This way when player object is destroyed the rank (enumeration) object is destroyed alongside.

Suggestion: UpdatePlayerTTT, updatePlayerCheckers, updatePlayerC4 should be moved in to their respective classes rather than in the super class. This ensures that each instance class does not have to implement methods that are irrelevant to those classes.

Suggestion: LeaderBoard should have an abstract label that displays that the class cannot be instantiated and needs to be extended through personalized classes (as shown below).

Suggestion: Ideally enumeration classes are used to set status by a predefined value. These ranks should be upper caps to align with format of constants. Additionally, these ranks can have predefined value assignment for easier tracking and comparison.

Suggestion: As discussed in the game logic section, ideally these games can extend the game class, given game class is an abstract class. This would allow for similar functionality while modifying the game logic individually.

For sound effect

Text